Name:- Suraj Singh Chauhan

Report: Optimising NYC Taxi Operations

Include your visualisations, analysis, results, insights, and outcomes. Explain your methodology and approach to the tasks. Add your conclusions to the sections.

1. Data Preparation

1.1. Loading the dataset

1.1.1. Sample the data and combine the files

Sampling:-

1. **The file is loaded in the memory**
2. Pickup date time are converted into date time format
3. Created new columns date and hour
4. Sample data grouped by date and hour
5. Sample of 5% of trips from each hour of each day.
6. Sample data is appended to data frame
7. Then data is stored to parquet file for faster loading (.csv is taking too much time to load the data)

Implementation:-

```
from google.colab import drive
drive.mount('/content/drive')

# Take a small percentage of entries from each hour of every date.
# Iterating through the monthly data:
#   read a month file -> day -> hour: append sampled data -> move to next hour -> move to next day after 24 hours -> move to next month file
# Create a single dataframe faor the year combining all the monthly data

# Select the folder having data files
import os

# Select the folder having data files
os.chdir('/content/drive/MyDrive/content/Assignments/EDA/data_NYC_Taxi/trip_records')

# Create a list of all the twelve files to read
```

```
file_list = os.listdir()

# initialise an empty dataframe
df = pd.DataFrame()

# iterate through the list of files and sample one by one:
for file_name in file_list:
    try:
        file_path = os.path.join(os.getcwd(), file_name)

        # Reading the current file
        if file_name.endswith(".parquet"):
            month_df = pd.read_parquet(file_path)
        elif file_name.endswith(".csv"):
           month_df = pd.read_csv(file_path)
        else:
          continue

        month_df['tpep_pickup_datetime'] =
pd.to_datetime(month_df['tpep_pickup_datetime'])
        month_df['date'] = month_df['tpep_pickup_datetime'].dt.date
        month_df['hour'] = month_df['tpep_pickup_datetime'].dt.hour

        # We will store the sampled data for the current date in this df by
appending the sampled data from each hour to this
        # After completing iteration through each date, we will append this
data to the final dataframe.
        sampled_data = month_df.groupby(['date', 'hour'],
group_keys=False).sample(frac=0.05, random_state=42)

        # Iterate through each date and hour

        # Loop through dates and then loop through every hour of each date

            # Iterate through each hour of the selected date

                # Sample 5% of the hourly data randomly

                # add data of this hour to the dataframe

        # Concatenate the sampled data of all the dates to a single dataframe
        df = pd.concat([df, sampled_data], ignore_index=True)
```

```
    except Exception as e:
        print(f"Error reading file {file_name}: {e}")

df=df.reset_index(drop=True)
df.head()
```

Analysis:-

The 2023 NYC Yellow Taxi dataset was loaded from 12 monthly Parquet files. Loading all 12 files at once would use up a lot of memory because each month has about 3 million rows. To handle this, I have taken a stratified sampling method was used. For each month's file, the pickup time was broken down into date and hour. Data was then grouped by date and hour, and 5% of the trips were randomly selected from each hour group. A fixed random seed (42) was used to make sure the results are the same every time. This method made sure that all times of day were represented equally across all months. All the sampled data from each month was combined into one big table with about 2 million rows, and then saved as a Parquet file called sample_taxi_data.parquet for quicker loading later.

2. Data Cleaning

2.1. Fixing Columns

2.1.1. Fix the index

```
df=df.reset_index(drop=True)
df.head()

if 'airport_fee' in df.columns and 'Airport_fee' in df.columns:
    df['airport_fee'] = df['airport_fee'].fillna(0) +
df['Airport_fee'].fillna(0)
    df.drop(columns=['Airport_fee'], inplace=True)
    print("Columns combined successfully.")


# check where values of fare amount are negative
(df['fare_amount'] < 0).sum()
print(df[df['fare_amount'] < 0][['fare_amount', 'RatecodeID',
'trip_distance']])
```

```
# Analyse RatecodeID for the negative fare amounts
df.loc[df['fare_amount'] < 0, 'RatecodeID'].value_counts()
# Find which columns have negative values
df.select_dtypes(include=['number']).lt(0).any()

# fix these negative values
df = df[(df['fare_amount'] >= 0) & (df['total_amount'] >= 0)]

# removed all the negative values
df.select_dtypes(include=['number']).lt(0).any()
```

Analysis:
The default integer index was reset by using df.reset_index(drop=True) and to create a clean sequential index starting from 0. This step is necessary for sampling and concatenating multiple monthly Data Frames, as the original indices from each file may overlap or be non-sequential. Also no additional columns were dropped at this stage except for those identified as redundant, such as the unnamed index columns Unnamed: 0 and Unnamed: 0.1 generated during Parquet export-import cycles. I combined the two airport_fee columns. The data set contained 2 columns and combined the both the columns in the dataset.

2.2. Handling Missing Values

2.2.1. Find the proportion of missing values in each column

```
# Find the proportion of missing values in each column
missing_val=(df.isna().sum() / len(df) * 100)
missing_val
```

Analysis:
Some columns had missing values, but only a small number of rows were affected. The passenger_count, RatecodeID, store_and_fwd_flag, and congestion_surcharge columns each had about 3.4% of their data missing, which was roughly 69,000 rows. And All other columns with numbers or dates had no missing data. The missing values appeared in these columns together, which suggests the issue might come from a particular vendor or part of the data process. None of the columns that show money, like fare_amount or tip_amount, had any missing values.

2.2.2. Handling missing values in passenger_count

```

# Display the rows with null values
```

```
# Impute NaN values in 'passenger_count'
null_val = (df['passenger_count'].isna().sum())
impute_val =
(df['passenger_count'].fillna(df['passenger_count'].median()))
impute_val
```

Analysis:
The missing values in the passenger_count column (about 3.4%) were filled by using the median value, which is 1.0 passenger. And the median was chosen instead of the average because it is less affected by extreme values. Then, I chose rows where passenger_count was 0 were removed because those were probably mistakes (no trip can have zero passengers). Also, where rows with passenger_count higher than 6 were removed since these were very rare and likely incorrect. After these steps, I have completed with the final dataset that it contain 1,935,290 rows with passenger counts that are between 1 and 6.

2.2.3. Handle missing values in RatecodeID

```
# Fix missing values in 'RatecodeID'
missing_val_rateID = (df['RatecodeID'].isna().sum())
df['RatecodeID'] = df['RatecodeID'].fillna(1)
```

Analysis:

The missing values in the RatecodeID field (about 3.4%) were filled with the number 1, that means the Standard rate. Also this is the most common rate code, and it makes sense to use it as a default when the rate is unknown. Since RatecodeID is a category with specific values from 1 to 6, using the most common value to fill in missing ones is the right approach.

2.2.4. Impute NaN in congestion_surcharge

```
# handle null values in congestion_surcharge
df['congestion_surcharge'].isnull().sum()
df['congestion_surcharge'] = df['congestion_surcharge'].fillna(0)
df['congestion_surcharge']
```

Analysis:

The missing values in the congestion_surcharge field was replaced with 0, that means no surcharge was applied. That makes sense because the congestion surcharge is a fixed $2.50 charge that either applies or doesn't. Also, If a value is missing, it most likely means the surcharge wasn't used for that trip. After filling in the missing values, the data

showed a bimodal pattern: most trips had a $2.50 surcharge (about 92.3% of trips), and the rest had $0.

2.3. Handling Outliers and Standardising Values

2.3.1. Check outliers in payment type, trip distance and tip amount columns

```
# remove passenger_count > 6
print(f"length: {len(df)}")
(df['passenger_count'] > 6).sum()
df = df[(df['passenger_count'] <= 6)]
df['passenger_count'].value_counts().sort_index()

# Continue with outlier handling
print(f"before length: {len(df)}")
df = df[df['trip_distance'] <= 150]
df = df[~((df['trip_distance'] == 0) & (df['fare_amount'] == 0))]
print(f"after length: {len(df)}")
df['trip_duration'] = (df['tpep_dropoff_datetime'] -
df['tpep_pickup_datetime']).dt.total_seconds() / 60
print(f"\nData types:\n{df.dtypes}")

# Do any columns need standardising?
# print(df.info())
if 'Unnamed: 0' in df.columns:
    df = df.drop(columns=['Unnamed: 0'])

if 'passenger_count' in df.columns:
    df=df[df['passenger_count'] > 0]
print('Data cleaned. Shape:', df.shape)
print('Missing values per column:')
print(df.isnull().sum())
```

Analysis:

When looking at the data using descriptive statistics I have seen some problems. The trip_distance had a maximum of 143,926 miles, which is clearly wrong. The fare_amount reached $1,117, which is also high. Rows with trip_distance over 150 miles were removed. Also, many entries where both trip_distance and fare_amount were 0 and also deleted. This left 1,966,594 rows. Rows with more than 6 passengers or zero passengers were also removed. And any rows with negative numbers in the monetary fields (extra, mta_tax, improvement_surcharge,

total_amount, congestion_surcharge) were filtered out because these should not be negative. After checking all the data, the final dataset had 1,935,290 rows and 23 columns with no missing data left.

3. Exploratory Data Analysis

3.1. General EDA: Finding Patterns and Trends

3.1.1. Classify variables into categorical and numerical

```
# Find and show the hourly trends in taxi pickups
hourly_trend =
df.groupby(df['tpep_pickup_datetime'].dt.hour)['tpep_pickup_datetime'].
size()
print(hourly_trend)
plt.figure(figsize=(12, 5))
plt.plot(hourly_trend.index, hourly_trend, marker='o', linewidth=2)
plt.title('Hourly Taxi Pickups(2023)')
plt.xlabel('Hour of the Day')
plt.ylabel('Number of Pickups')
plt.xticks(range(0,24))
plt.grid(True)
plt.show()
```

Analysis:

The categorical variables include: VendorID, RatecodeID, tpep_pickup_datetime, tpep_dropoff_datetime, PULocationID, DOLocationID, payment_type. Numerical variables include: passenger_count, trip_distance, fare_amount, trip_duration. Monetary parameters (which are all numerical and continuous): fare_amount, extra, mta_tax, tip_amount, tolls_amount, improvement_surcharge, total_amount, congestion_surcharge, airport_fee.

3.1.2. Analyse the distribution of taxi pickups by hours, days of the week, and months

```
# Find and show the daily trends in taxi pickups (days of the week)
df['pickup_day'] = df['tpep_pickup_datetime'].dt.day_name()
df['pickup_week'] = df['tpep_pickup_datetime'].dt.dayofweek
daily_trend =
df.groupby(['pickup_week','pickup_day']).size().reset_index(name='pickups'
)
print(daily_trend)
```

```
plt.figure(figsize=(12, 5))
plt.bar(range(7), daily_trend['pickups'])
plt.xlabel('Day of the Week')
plt.ylabel('Number of Pickups')
plt.title('Daily Taxi Pickups(2023)')
plt.xticks(range(7), daily_trend['pickup_day'], rotation=45)
# plt.grid(True)
plt.show()

# Show the monthly trends in pickups
df['pickup_month'] = df['tpep_pickup_datetime'].dt.month
monthly_trend =
df.groupby(['pickup_month']).size().reset_index(name='pickups')
month_name = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug',
'Sep', 'Oct', 'Nov', 'Dec']
monthly_trend['month_name'] = monthly_trend['pickup_month'].apply(lambda
x: month_name[x-1])
print(monthly_trend)

plt.figure(figsize=(12, 5))
plt.bar(monthly_trend['month_name'], monthly_trend['pickups'],
color='green')
plt.xlabel('Month')
plt.ylabel('Number of Pickups')
plt.title('Monthly Taxi Pickups(2023)')
plt.xticks(rotation=45)
plt.show()

# Show the monthly trends in pickups
df['pickup_month'] = df['tpep_pickup_datetime'].dt.month
monthly_trend =
df.groupby(['pickup_month']).size().reset_index(name='pickups')
month_name = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug',
'Sep', 'Oct', 'Nov', 'Dec']
monthly_trend['month_name'] = monthly_trend['pickup_month'].apply(lambda
x: month_name[x-1])
print(monthly_trend)

plt.figure(figsize=(12, 5))
plt.bar(monthly_trend['month_name'], monthly_trend['pickups'],
color='green')
plt.xlabel('Month')
```

```
plt.ylabel('Number of Pickups')
plt.title('Monthly Taxi Pickups(2023)')
plt.xticks(rotation=45)
plt.show()
```

Analysis:

When hourly analysis showed a clear pattern of two busy times: trips were lowest in the early morning with the fewest at 4 AM, around 10,169 trips. Then, I seen that trips went up quickly during the morning rush, reaching their highest point at 6 PM, with 136,891 trips. There was also a smaller peak around 10-11 AM. While looking at the whole day, Thursday was the busiest weekday with 303,673 trips, and Monday was the quietest with 242,060 trips. Whereas, on weekends, the number of trips was slightly less, but the times when people traveled were different. And for the whole month, March and May were the busiest months with 178,504 and 174,675 trips, respectively. August and July were the slowest, with 140,267 and 144,496 trips, likely because of summer vacations when more New Yorkers travel out of the city.

3.1.3. Filter out the zero/negative values in fares, distance and tips

```
# Create a df with non zero entries for the selected parameters.
df_nonzero = df[(df['fare_amount'] > 0) & (df['total_amount'] > 0)].copy()
print(f"Dataset: {len(df)}")
print(f"Rows: {len(df_nonzero)}")
print(f"Zero values removed: {len(df) - len(df_nonzero)}")

df_tips = df_nonzero[df_nonzero['tip_amount'] > 0].copy()
print(f"\nTips dataset: {len(df_tips)}")
print(f"Zero values removed: {len(df_nonzero) - len(df_tips)}")
print(df_nonzero[['fare_amount', 'tip_amount', 'total_amount',
'trip_distance']].describe())
```

Analysis:

When looking at the rows where the fare and total amount were more than zero, I made a data set called df_nonzero with 1,935,014 rows. There were 276 rows where the fare was zero and 102 rows where the total amount was zero. Another data set called df_tips was made by only including rows where the tip amount was greater than zero, resulting in 1,503,920 rows. And the reason some trips had zero distance was

because sometimes people picked up and dropped off in the same area, which is normal. Also, then zero tips are fine for cash payments since tips aren't recorded electronically.

3.1.4. Analyse the monthly revenue trends

```
# Group data by month and analyse monthly revenue
month_revenue = df_nonzero.groupby('pickup_month').agg({
    'total_amount': ['sum', 'mean', 'count']
}).round(2)
month_revenue.columns = ['Total revenue', 'Avg rate', 'Num trips']
month_revenue=month_revenue.reset_index()
month_name = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug',
'Sep', 'Oct', 'Nov', 'Dec']
month_revenue['month_name'] = month_revenue['pickup_month'].apply(lambda
x: month_name[x-1])
print(month_revenue[['month_name', 'Total revenue', 'Avg rate', 'Num
trips']])

#total revenue
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].bar(month_revenue['month_name'], month_revenue['Total revenue'])
axes[0].set(title='Month total revenue (2023)', xlabel='Month',
ylabel='Total revenue($ millions)')
axes[0].tick_params(axis='x', rotation=45)

axes[1].plot(month_revenue['month_name'], month_revenue['Avg rate'],
marker='o', linewidth=2)
axes[1].set(title='Month average rate (2023)', xlabel='Month',
ylabel='Average ($)')
axes[1].tick_params(axis='x', rotation=45)
axes[1].grid()

plt.tight_layout()
plt.show()
```

Analysis:

When looking at monthly revenue, I checked that May had the highest total amount at $5.15 million, followed by March at $5.06 million and October at $5.15 million. And the slowest months were August with $4.11 million and July with $4.22 million. Even though July and August had

fewer trips, the average fare per trip went up slightly, from about $27-28 in January and February to about $29-30 in summer months. That means the rides were longer or more expensive. So, The drop in total revenue during summer months suggests that fewer people were using taxis, likely due to seasonal travel and less commuting.

3.1.5. Find the proportion of each quarter's revenue in the yearly revenue

```
# Calculate proportion of each quarter
df_nonzero['quater'] = df_nonzero['pickup_month'].apply(lambda x: (x-
1)//3+1)
quater_revenue=df_nonzero.groupby('quater')['total_amount'].sum()
quater_label = ['Q1 (jan-mar)', 'Q2 (apr-jun)', 'Q3 (jul-sep)', 'Q4 (oct-
dec)']
for quater in range(1,5):
  revenue=quater_revenue[quater]
  print(f"{quater_label[quater-1]}: ${revenue/1e6:.2f}M")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].bar(quater_label, quater_revenue/1e6)
axes[0].set(title='Quarter total revenue (2023)', ylabel='Total revenue($
millions)')

axes[1].pie(quater_revenue, labels=quater_label, autopct='%1.1f%%')
axes[1].set(title='Quarterly revenue(2023)')

plt.tight_layout()
plt.show()
```

Analysis:

The second quarter, which includes April to June, had the highest revenue at $14.93 million, making up 26.6% of the total. The fourth quarter, from October to December, had $14.78 million, or 26.3%. The first quarter, from January to March, had $13.68 million, or 24.4%, and the third quarter, from July to September, was the lowest with $12.74 million, or 22.7%.I checked that the revenue across quarters was pretty balanced, with no one quarter taking over. The dip in Q3 matches was the lower number of trips seen in summer months. This shows that taxi demand in NYC is stable all year, though Q2 and Q4 had slight peaks due to spring and fall commuting seasons.

3.1.6. Analyse and visualise the relationship between distance and fare amount

```
# Show how trip fare is affected by distance

df_trip_fare = df_nonzero[df_nonzero['trip_distance'] > 0].copy()
correlation =
df_trip_fare['trip_distance'].corr(df_trip_fare['fare_amount'])

plt.figure(figsize=(12, 5))
plt.scatter(df_trip_fare['trip_distance'], df_trip_fare['fare_amount'])
plt.xlabel('Trip Distance')
plt.ylabel('Fare Amount')
plt.title('Trip Distance vs Fare Amount')
plt.xlim(0,50)
plt.ylim(0,150)

# trend line
z=np.polyfit(df_trip_fare['trip_distance'], df_trip_fare['fare_amount'],
1)
p=np.poly1d(z)
x_axis=np.linspace(0,50,100)
plt.plot(x_axis, p(x_axis),"r-", linewidth=2, label=f'Trend: Fare=
{z[0]:.2f}*Distance + {z[1]:.2f}')
plt.legend()
plt.show()

print(f"Correlation: {correlation}")
```

Analysis:

I have seen that there was a strong positive link between trip distance and fare amount, with a correlation of 0.9445. This means distance is the main thing that determines how much the fare is. When I made a line showing the trend from the data (only looking at distances up to 50 miles and fares up to $150), the equation was: Fare = 3.10 * Distance + 9.17. That means there's a base fare of about $9.17 and a rate of about $3.10 per mile. Some differences from the line are because of things like rush hour charges and waiting time in traffic, and trips that have a flat rate, like to JFK airport.

3.1.7. Analyse the relationship between fare/tips and trips/passengers

```
# Show relationship between fare and trip duration

correlation_duration =
df_nonzero['trip_duration'].corr(df_nonzero['fare_amount'])
plt.figure(figsize=(12, 5))

filter_data = df_nonzero[(df_nonzero['trip_duration'] > 0) &
                         (df_nonzero['trip_duration'] <= 100) &
                         (df_nonzero['fare_amount'] <= 150)]

plt.scatter(filter_data['trip_duration'], filter_data['fare_amount'],
alpha=0.3, s=10)
plt.xlabel('Fare Amount')
plt.ylabel('Trip Duration')
plt.title('Fare Amount vs Trip Duration')

z=np.polyfit(filter_data['trip_duration'], filter_data['fare_amount'],1)
p=np.poly1d(z)
x_axis=np.linspace(0,100,100)
plt.plot(x_axis, p(x_axis),"r-", linewidth=2, label=f'Trend: Fare=
{z[0]:.2f}*Distance + {z[1]:.2f}')
plt.legend()
plt.show()

print(f"Correlation: {correlation_duration}")

# Show relationship between fare and number of passengers
correlation_passenger =
df_nonzero['passenger_count'].corr(df_nonzero['fare_amount'])

fare_passenger = df_nonzero.groupby('passenger_count').agg({
    'fare_amount': ['mean', 'median', 'count']
}).round(2)

fare_passenger.columns = ['Mean fare', 'Median fare', 'Num trips']
print(fare_passenger)

fig, axes = plt.subplots(1,2, figsize=(14, 5))

data_plot=[df_nonzero[df_nonzero['passenger_count'] ==
i]['fare_amount'].values
```

```
           for i in range(1,7)]
axes[0].boxplot(data_plot, label=range(1,7))
axes[0].set(xlabel='Passenger count', ylabel='Fare amount', title='Fare
amount by passenger count')

avg_fare_passenger=[df_nonzero[df_nonzero['passenger_count']==i]['fare_amo
unt'].mean()
for i in range(1,7)]
axes[1].bar(range(1,7), avg_fare_passenger)
axes[1].set(xlabel='Passenger count', ylabel='Average fare amount',
title='Average fare amount by passenger count')
axes[1].tick_params(axis='x')

plt.tight_layout()
plt.show()

print(f"\nCorrelation: {correlation_passenger}")


# Show relationship between tip and trip distance
# Show relationship between fare and trip duration

correlation_tip_dist =
df_tips['trip_distance'].corr(df_tips['tip_amount'])
plt.figure(figsize=(12, 5))

filter_data = df_tips[(df_tips['trip_distance'] > 0) &
                      (df_tips['trip_distance'] <= 30) &
                      (df_tips['tip_amount'] <= 50)]

plt.scatter(filter_data['trip_distance'], filter_data['tip_amount'],
alpha=0.3, s=10)
plt.xlabel('Trip Distance')
plt.ylabel('Tip Amount')
plt.title('Tip Amount vs Trip Distance')

z=np.polyfit(filter_data['trip_distance'], filter_data['tip_amount'],1)
p=np.poly1d(z)
x_axis=np.linspace(0, 30, 100)
plt.plot(x_axis, p(x_axis),"r-", linewidth=2, label=f'Trend: Tip=
{z[0]:.2f}*Distance + {z[1]:.2f}')
plt.legend()
plt.tight_layout()
```

```
plt.show()

print(f"Correlation: {correlation_tip_dist}")
```

Analysis:

When looking at fare and trip duration, I have seen that there was a moderate positive link with a correlation of 0.27. That means longer trips tend to cost more, but distance has a bigger effect on fare than time alone. Also Fare vs passenger count showed a very weak link (0.046), indicating that fare doesn't change much based on how many people are in the cab. However, trips with four passengers had the highest average fare ($24.08), probably because group trips are longer. And the Tip vs trip distance had a moderate positive link (0.78), meaning passengers on longer trips gave more tips. The trend line showed Tip = 0.43 * Distance + 1.86.

3.1.8. Analyse the distribution of different payment types

```
# Analyse the distribution of different payment types (payment_type).
payment_types = {
    0: 'Unknown',
    1: 'Credit Card',
    2: 'Cash',
    3: 'No charge',
    4: 'Dispute',
    5: 'Unknown',
}

payment_dist = (df.groupby('payment_type')['total_amount']
                .agg(['count', 'sum', 'mean'])
                .rename(columns={'count':'Num Trips', 'sum': 'Total
revenue', 'mean': 'Avg rate'})
                .round(2))

payment_dist['payment_type'] = payment_dist.index.map(lambda x:
payment_types.get(int(x), 'Unknown'))
payment_dist['percentage'] = (payment_dist['Num Trips'] /
payment_dist['Num Trips'].sum() * 100).round(1)
payment_dist = payment_dist.sort_values('Num Trips', ascending=False)
print(payment_dist[['payment_type', 'Num Trips', 'percentage', 'Total
revenue', 'Avg rate']].to_string())

fig, ax = plt.subplots(2, 2, figsize=(14, 10))
```

```
labels = payment_dist['payment_type']

ax[0,0].bar(labels, payment_dist['Num Trips'])
ax[0,0].set(title='Dustribution of payment type', ylabel='Number of
trips')
ax[0,0].tick_params(axis='x', rotation=45)

ax[0,1].pie(payment_dist['Num Trips'], labels=labels, autopct='%1.1f%%',
startangle=90)
ax[0,1].set(title='Payment type distribution (percentage)')

ax[1,0].bar(labels, payment_dist['Total revenue']/1e6, color='red')
ax[1,0].set(title='Dustribution of payment type', ylabel='Total revenue($
millions)')
ax[1,0].tick_params(axis='x', rotation=45)

ax[1,1].bar(labels, payment_dist['Avg rate'], color='green')
ax[1,1].set(title='Dustribution of payment type', ylabel='Average rate
($)')
ax[1,1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```

Analysis:

I checked that credit card was the most common payment method, used in 81.6% of trips (1,578,570 trips), bringing in $47.16 million in revenue with an average fare of $29.88. The cash was used in 17.2% of trips (333,404 trips) with a lower average fare of $25.21. That is because cash payments don't include electronic tips and are usually for shorter, local rides. And the disputes made up 0.7% of trips and No charge was 0.5%. The high use of credit cards shows that more people are using digital payments, which impacts tip analysis because only credit card tips are recorded electronically.

3.1.9. Load the taxi zones shapefile and display it

```
# import geopandas as gpd
import geopandas as gpd

# Read the shapefile using geopandas
zones_path =
"/content/drive/MyDrive/content/Assignments/EDA/data_NYC_Taxi/taxi_zones/t
axi_zones.shp"
zones = gpd.read_file(zones_path)
```

```
zones.head()

print(zones.info())
zones.plot()
```

Analysis:

Here after looking at the zones path data, that is the taxi zones shapefile (taxi_zones.shp) was loaded using GeoPandas, making a GeoDataFrame with 263 rows (one for each taxi zone) and 7 columns: OBJECTID, Shape_Leng, Shape_Area, zone, LocationID, borough, and geometry. The LocationID numbers go from 1 to 263 and help me to match with trip data. These zones cover six boroughs: Manhattan, Queens, Brooklyn, Bronx, Staten Island, and EWR (Newark). When I plotted the GeoDataFrame, I got an outline map showing all taxi zones, and it showed that Manhattan has a lot of zones compared to the outer boroughs.

3.1.10. Merge the zone data with trips data

```
# Merge zones and trip records using locationID and PULocationID

zones_PU = zones[['LocationID', 'zone', 'borough', 'geometry']].copy()
zones_PU.rename(columns={'LocationID': 'PULocationID', 'zone': 'pu_zone',
'borough': 'pu_borough'}, inplace=True)

df_zones = df.merge(zones_PU, on='PULocationID', how='left')
print(df_zones[['PULocationID', 'pu_zone', 'pu_borough',
'total_amount']].head(10))
```

Analysis:

The zones GeoDataFrame was combined with the trips data set by matching PULocationID (from the trip data) with LocationID (from the zones). I have added zone names and boroughs, renaming them to pu_zone and pu_borough for clarity. The result, called df_zones, kept all 1,935,412 trip rows with the added zone information. Sample results confirmed that the matches were correct, like JFK Airport (LocationID=132), Upper East Side South (237), and Midtown Center (161) showing up among the top pickup zones.

3.1.11. Find the number of trips for each zone/location ID

```
# Group data by location and calculate the number of trips
```

```
data_by_location=df_zones.groupby('PULocationID').agg({
    'total_amount': ['count'],
    'pu_zone': 'first',
    'pu_borough': 'first'
}).round(2)

data_by_location.columns = ['Num_trips', 'Zone', 'Borough']
data_by_location = data_by_location.reset_index()
data_by_location = data_by_location.sort_values('Num_trips',
ascending=False)
print(data_by_location)
```

Analysis:

When grouping df_zones by PULocationID showed that JFK Airport (LocationID 132) had the most pickups, with 103,710 trips, followed by Upper East Side South (92,041), Midtown Center (90,260), Upper East Side North (80,851), and Midtown East (69,789). LaGuardia Airport was sixth with 68,255 trips. On the other end, zones like Rikers Island, Oakwood, Great Kills, and Crotona Park had only one trip each, indicating very little taxi activity in those areas. A total of 260 different zones had at least one recorded pickup.

3.1.12. Add the number of trips for each zone to the zones dataframe

```
# Merge trip counts back to the zones GeoDataFrame
zone_with_trips = zones.merge(data_by_location[['PULocationID',
'Num_trips']],
                              left_on='LocationID',
right_on='PULocationID', how='left')
zone_with_trips['Num_trips']=
zone_with_trips['Num_trips'].fillna(0).astype(int)

print(zone_with_trips.nlargest(10, 'Num_trips')[['zone', 'LocationID',
'Num_trips']])
```

Analysis:

Trip counts were added back to the zones GeoDataFrame through a left join on LocationID, creating zone_with_trips. Missing trip counts for zones with no pickups were filled with 0. This enriched GeoDataFrame includes both geographic information and trip volume, making it possible to create a choropleth map. The top 10 zones by trip count were: JFK Airport

(103,710), Upper East Side South (92,041), Midtown Center (90,260), Upper East Side North (80,851), Midtown East (69,789), LaGuardia Airport (68,255), Penn Station/Madison Sq West (67,717), Times Sq/Theatre District (64,951), Lincoln Square East (63,371), and Murray Hill (57,837).

3.1.13. Plot a map of the zones showing number of trips

```
from matplotlib import legend
# Define figure and axis
fig, ax = plt.subplots(1, 1, figsize = (10, 8))

# Plot the map and display it
zone_with_trips.plot(column='Num_trips', ax=ax, cmap='YlGn',
edgecolor='black', legend=True, legend_kwds={'label': "Number of trips",
'orientation': "vertical"})
plt.title('NYC Taxi Zones')
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.show()
```

Analysis:

A choropleth map was created using the YIGn (Yellow-Green) color scheme. And while ploting the map clearly shows that Manhattan areas, especially Midtown and the Upper East Side, have the highest number of trips (shown in the darkest green). Airport zones like JFK and LaGuardia in Queens are also busy spots. The outer boroughs such as the Bronx, Staten Island, and most of Brooklyn have much fewer trips (shown in light yellow). This shows me that the concentration of trips in Manhattan is due to high levels of business and tourism activity, and it helps to determine where taxi services should focus.

3.1.14. Conclude with results

```
# can you try displaying the zones DF sorted by the number of trips?
zones_sorted = zone_with_trips[["LocationID", "zone", "borough",
"Num_trips"]].sort_values(by="Num_trips",
ascending=False).reset_index(drop=True)
print("Zones sorted by number of trips (Total zones: " +
str(len(zones_sorted)) + ")")
print(zones_sorted.to_string())
```

Analysis:

The general data exploration found several key points: (1) The busiest times are between 5-8 PM, with the peak at 6 PM (136,891 trips recorded). The quietest time is between 3-5 AM. (2) Thursdays and Wednesdays are the busiest days of the week, while Mondays and Sundays are the quietest. (3) The most trips and revenue happen in spring (March and May) and fall (October), while summer (July and August) has fewer trips. (4) The distance of a trip is the main factor affecting the fare (correlation of 0.94). (5) Most trips (81.6%) are paid for using a credit card. (6) Manhattan, especially Midtown and the Upper East Side, has the most trips, and JFK and LaGuardia are the top zones in the outer boroughs.

3.2. Detailed EDA: Insights and Strategies

3.2.1. Identify slow routes by comparing average speeds on different routes

```
# Find routes which have the slowest speeds at different times of the day
df_route_speed = df_nonzero[(df_nonzero['trip_distance'] > 0) &
(df_nonzero['trip_duration'] > 0)].copy()
df_route_speed['speed'] = (df_route_speed['trip_distance'] /
df_route_speed['trip_duration'] *60).clip(0,100)
df_route_speed['pickup_hour'] =
df_route_speed['tpep_pickup_datetime'].dt.hour
df_route_speed['route'] = df_route_speed['PULocationID'].astype(str) + " -
" + df_route_speed['DOLocationID'].astype(str)

slowest_route = df_route_speed.groupby(['pickup_hour', 'route']).agg({
    'speed':['mean', 'count'],
    'trip_distance': 'mean'
}).round(2)
slowest_route.columns = ['Avg speed', 'Num trips', 'Avg distance']
slowest_route = slowest_route.reset_index()
slowest_route = slowest_route[slowest_route['Num trips'] >= 10]
print('\nslowest route: ')
print(slowest_route)

slowest_10 = slowest_route.nsmallest(10, 'Avg speed')[['route',
'pickup_hour', 'Avg speed', 'Num trips', 'Avg distance']]
print(print('\nslowest 10: '))
print(slowest_10)
```

```
speed_by_hour =
df_route_speed.groupby('pickup_hour')['speed'].mean().round(2)

print('\nspeed by hour:')
print(speed_by_hour)
```

Analysis:

When speed was calculated by dividing trip distance by trip time (converted to hours) and multiplying by 60 to get speed in miles per hour. Speeds were capped at 100 mph to exclude unusual values. Also the routes were defined as pairs of pickup and drop-off location IDs. And routes with at least 10 trips were analyzed; a total of 36,685 route-hour combinations were used. The slowest route found was from zone 100 to 100 (within the same zone) during hour 13, with an average speed of just 3.04 mph. When short trips within Manhattan during midday (11 AM to 3 PM) were the slowest (3 to 7 mph), matching the heavy traffic in Midtown. Longer routes to airports were generally faster (15 to 25 mph), as they used highways. Identifying these slow routes helps dispatchers avoid them during peak hours and find better routes.

3.2.2. Calculate the hourly number of trips and identify the busy hours

```
# Visualise the number of trips per hour and find the busiest hour
hourly_trip = df.groupby('hour').size().reset_index(name='trips')
busy_hour = hourly_trip.loc[hourly_trip['trips'].idxmax()]

plt.figure(figsize=(14,6))
bars = plt.bar(hourly_trip['hour'], hourly_trip['trips'], color='skyblue')
bars[int(busy_hour['hour'])].set_color('red')
plt.xlabel('Hour of the day')
plt.ylabel('Number of trips')
plt.title('Number of trips per hour')
plt.xticks(range(0,24))

for i, (hour, trips) in enumerate(zip(hourly_trip['hour'],
hourly_trip['trips'])):
    plt.text(i, trips, str(int(trips)), ha='center', va='bottom',
fontsize=8)
plt.tight_layout()
plt.show()
```

Analysis:

The hourly trip data showed a clear peak in the evening. Hour 18 (6 PM) had the highest number of trips at 136,891, followed by hour 17 (5 PM) and hour 19 (7 PM). The morning also saw activity starting from hour 9 and increasing throughout the day. The bar chart highlighted hour 18 in red as the busiest time. The quietest time was hour 4 (4 AM) with 10,169 trips. This pattern of two peaks, with the evening peak being the main one, shows typical urban traffic and activity related to dinner and nightlife.

3.2.3. Scale up the number of trips from above to find the actual number of trips

```
# Visualise the number of trips per hour and find the busiest hour
hourly_trip = df.groupby('hour').size().reset_index(name='trips')
busy_hour = hourly_trip.loc[hourly_trip['trips'].idxmax()]

plt.figure(figsize=(14,6))
bars = plt.bar(hourly_trip['hour'], hourly_trip['trips'], color='skyblue')
bars[int(busy_hour['hour'])].set_color('red')
plt.xlabel('Hour of the day')
plt.ylabel('Number of trips')
plt.title('Number of trips per hour')
plt.xticks(range(0,24))

for i, (hour, trips) in enumerate(zip(hourly_trip['hour'],
hourly_trip['trips'])):
    plt.text(i, trips, str(int(trips)), ha='center', va='bottom',
fontsize=8)
plt.tight_layout()
plt.show()
```

Analysis:

Since the data was collected with a 5% sample rate, actual trip numbers were estimated by multiplying the sampled count by 20. The top five busiest hours in actual trips were: Hour 18 (6 PM): about 2,737,820 trips, Hour 17 (5 PM): about 2,617,180 trips, Hour 19 (7 PM): about 2,456,240 trips, Hour 16 (4 PM): about 2,415,380 trips, and Hour 15 (3 PM): about 2,409,040 trips. This scaling clearly shows that the evening rush hours (3-8 PM) are responsible for a large portion of taxi demand in New York City all year.

3.2.4. Compare hourly traffic on weekdays and weekends

```
# Compare traffic trends for the week days and weekends
df['is_weekend'] = df['pickup_week'].isin([5,6])

hourly_day_type = df.groupby(['hour',
'is_weekend']).size().unstack(fill_value=0)
hourly_day_type.columns = ['Weekday', 'Weekend']

plt.figure(figsize=(14, 6))
plt.plot(hourly_day_type.index, hourly_day_type['Weekday'], marker='o',
label='Weekday', linewidth=2)
plt.plot(hourly_day_type.index, hourly_day_type['Weekend'], marker='o',
label='Weekend', linewidth=2)
plt.xlabel('Hour of the day')
plt.ylabel('Number of trips')
plt.title('Hourly Traffic Trends')
plt.xticks(range(0, 24))
plt.legend()
plt.tight_layout()
plt.show()
```

Analysis:

When checking the weekday and weekend traffic patterns they are different. Weekdays show a quick increase in the morning starting at 6-7 AM, with a strong peak between 5-7 PM, reflecting commuter traffic. Weekends have a later and flatter morning increase (activity rises around 9-10 AM), with a broader and less intense evening peak. Weekend late-night hours (10 PM to 2 AM) have more trips than weekdays, driven by nightlife and social activities. That means cabs should be available later on weekends and stay active until midnight, while on weekdays, more cabs should be out early in the morning starting from 7 AM.

3.2.5. Identify the top 10 zones with high hourly pickups and drops

```
# Find top 10 pickup and dropoff zones
hourly_pickup_zone = df_zones.groupby(['hour',
'pu_zone']).size().unstack(fill_value=0)

top_10_pickup = df_zones['pu_zone'].value_counts().head(10).index.tolist()

plt.figure(figsize=(14, 6))
for zone in top_10_pickup:
```

```
  if zone in hourly_pickup_zone.columns:
    plt.plot(hourly_pickup_zone.index, hourly_pickup_zone[zone],
marker='o', label=zone, linewidth=2)

plt.xlabel('Hour of days')
plt.ylabel('Number of pickups')
plt.title('Hourly pickup for top 10 zones (2023)')
plt.xticks(range(0, 24))
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("\nTop 10 zones")
print(df_zones['pu_zone'].value_counts().head(10))
```

Analysis:

The top 10 pickup zones by total trips were: JFK Airport (103,710 trips), Upper East Side South (92,041 trips), Midtown Center (90,260 trips), Upper East Side North (80,851 trips), Midtown East (69,789 trips), LaGuardia Airport (68,255 trips), Penn Station/Madison Sq West (67,717 trips), Times Sq/Theatre District (64,951 trips), Lincoln Square East (63,371 trips), and Murray Hill (57,837 trips). Hourly pickup trends for these zones showed that airport zones like JFK and LaGuardia had steady demand throughout the day, while Manhattan business zones like Midtown Center and Upper East Side had the most trips between 5-8 PM. Penn Station had a big increase in pickups between 8-9 AM, matching the arrival of commuters from trains.

3.2.6. Find the ratio of pickups and dropoffs in each zone

```
# Find the top 10 and bottom 10 pickup/dropoff ratios
pu_counts=df_zones.groupby('pu_zone').size().reset_index(name='pickups')
pu_counts.columns = ['Zone', 'Pickups']

zones_do = zones[['LocationID', 'zone']].copy()
zones_do.rename(columns={'LocationID': 'DOLocationID', 'zone': 'do_zone'},
inplace=True)
df_with_zones = df_zones.merge(zones_do, on='DOLocationID', how='left')

do_counts =
df_with_zones.groupby('do_zone').size().reset_index(name='dropoffs')
do_counts.columns = ['Zone', 'Dropoffs']
```

```
zone_ratio = pd.merge(pu_counts, do_counts, on='Zone',
how='outer').fillna(0)

zone_ratio['Ratio'] = zone_ratio.apply(lambda row: row['Pickups'] /
row['Dropoffs'] if row['Dropoffs'] > 0 else float('nan'), axis=1)

zone_valid_ratio = zone_ratio.dropna(subset=['Ratio'])
zone_ratio = zone_valid_ratio.sort_values('Ratio',
ascending=False).reset_index(drop=True)

print("Top 10 zones: ")
print(zone_ratio.head(10))
print("\nBottom 10 zones: ")
print(zone_ratio.tail(10))
```

Analysis:

The pickup-to-dropoff ratio where I seen that where trips are coming from and going to. East Elmhurst had the highest ratio (8.09), meaning it has many more pickups than dropoffs. Also, this area is next to LaGuardia Airport and is a place where people leave from. JFK Airport (ratio 4.57) and LaGuardia Airport (ratio 2.89) also had high pickup ratios, showing they are key starting points. In contrast, zones like Bayside (0.058), Ridgewood (0.054), Greenpoint (0.051), and Douglaston (0.051) have more dropoffs than pickups, meaning they are main places where people arrive. Understanding these ratios is important to help move empty taxis to areas with lots of dropoffs after trips.

3.2.7. Identify the top zones with high traffic during night hours

```
# During night hours (11pm to 5am) find the top 10 pickup and dropoff
zones
# Note that the top zones should be of night hours and not the overall top
zones
hours = [23, 0, 1, 2, 3, 4]
df_night = df_zones[df_zones['hour'].isin(hours)]

night_pickups = df_night['pu_zone'].value_counts()

total_pickups = df_zones['pu_zone'].value_counts()

night_compare = pd.DataFrame({
```

```
'Total Pickups': total_pickups,
'Night Pickups': night_pickups,
'Night Pct': (night_pickups / total_pickups *100).round(2)
}).fillna(0).sort_values('Night Pickups', ascending=False)

print(night_compare.head(15))

print("Zone with high night traffic")
night_compare_high_pct = night_compare[night_compare['Night Pct'] >=
35].sort_values('Night Pct', ascending=False)
print(night_compare_high_pct.head(10))

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

night_compare.head(10)[['Night Pickups', 'Total
Pickups']].plot(kind='bar', ax=ax1)
ax1.set_xlabel('Zone')
ax1.set_ylabel('Number of trips')
ax1.set_title('Night traffic by zone')
ax1.legend(['Night', 'Total'])
plt.setp(ax1.xaxis.get_majorticklabels(), rotation=45, ha='right')

night_compare.head(15)['Night Pct'].plot(kind='barh', ax=ax2,
color='purple')
ax2.set_xlabel('Percentage')
ax2.set_ylabel('Zone')
ax2.set_title('Night traffic by zone (percentage)')

plt.tight_layout()
plt.show()
```

Analysis:

Night time was considered as from 11 PM until 5 AM (hours 23, 0, 1, 2, 3, 4). The areas that had the most night trips were: East Village (16,086 trips, 35.4% of total), JFK Airport (14,361; 13.9%), West Village (13,275; 30.7%), Clinton East (10,347; 20.1%), and Lower East Side (10,115; 51.7%). Other areas with a high number of night trips (more than 35% of their total trips happened at night) were Lower East Side (51.7%), East Village (35.4%), and Greenwich Village South (36.7%). These areas are known for their nightlife. Because of the high night trip activity, the availability of taxis during late hours in these areas can result in better usage of taxis than usual.

3.2.8. Find the revenue share for nighttime and daytime hours

```
# Filter for night hours (11 PM to 5 AM)
hours = [23, 0, 1, 2, 3, 4]
df_night = df[df['hour'].isin(hours)]
df_day =df[~df['hour'].isin(hours)]

night_reven = df_night['total_amount'].sum()
day_reven = df_day['total_amount'].sum()
total_reven = df['total_amount'].sum()

night_per = (night_reven / total_reven) *100
day_per = (day_reven / total_reven) *100

print("Revenue: Night Vs Day")
print(f"Night: ${night_reven:.2f}")
print(f"Day: ${day_reven:.2f}")
print(f"Total: ${total_reven:.2f}")
print(f"Night: {night_per:.2f}%")
print(f"Day: {day_per:.2f}%")
print(f"Night trip count: {len(df_night)}")
print(f"Day trip count: {len(df_day)}")
print(f"Night Avg fare: ${df_night['total_amount'].mean():.2f}")
print(f"Day Avg fare: ${df_day['total_amount'].mean():.2f}")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

ax1.pie([night_reven, day_reven],
        labels=['Night (11PM - 5AM)', 'Day (6AM - 10PM)'],
        autopct = '%1.1f%%',
        startangle=90)
ax1.set_title("Revenue Night Vs Day")

categories = ['Night', 'Day']
trips = [len(df_night), len(df_day)]
revenues = [night_reven/1000, day_reven/1000]

x=np.arange(len(categories))
width=0.35

ax2_twin = ax2.twinx()
bars1 = ax2.bar(x-width/2, trips, width=width, label='Trips')
bars2 = ax2_twin.bar(x+width/2, revenues, width=width, color='orange',
label='Revenue')
```

```
ax2.set_ylabel('Trips')
ax2_twin.set_ylabel('Revenue (thousands)')
ax2.set_title('Trips Vs Revenue')
ax2.set_xticks(x)
ax2.set_xticklabels(categories)

for bar in bars1:
  height = bar.get_height()
  ax2.text(bar.get_x() + bar.get_width()/2., height,
           f'{int(height)}', ha='center', va='bottom', fontsize=9)

for bar in bars2:
  height = bar.get_height()
  ax2_twin.text(bar.get_x() + bar.get_width()/2., height,
                f'{height:.0f}k', ha='center', va='bottom', fontsize=9)

ax2.legend(loc='upper left')
ax2_twin.legend(loc='upper right')

plt.tight_layout()
plt.show()
```

Analysis:

During the day, from 6 AM to 10 PM, the total revenue was $49.72M, which was 88.59% of all revenue, from 1,715,778 trips, with an average fare of $28.98. During night hours, from 11 PM to 5 AM, the revenue was $6.40M, making up 11.41% of all revenue, from 219,512 trips, with an average fare of $29.17. Although only 11.4% of the total trips were during the night, the average revenue per trip was slightly more than that during the day. This is likely because night trips could be longer or more expensive, such as trips to the airport or across different boroughs. So, even though there are fewer night trips, they can still generate good revenue and should not be ignored when planning for the taxi fleet.

3.2.9. For the different passenger counts, find the average fare per mile per passenger

```
# Analyse the fare per mile per passenger for different passenger counts

df_analysis = df[(df['trip_distance'] > 0) &
                 (df['total_amount'] > 0) &
                 (df['passenger_count'] > 0)].copy()
```

```
df_analysis['fare_per_mile'] = df_analysis['fare_amount'] /
df_analysis['trip_distance']
df_analysis['fare_per_mile_per_passenger'] = df_analysis['total_amount'] /
(df_analysis['trip_distance'] * df_analysis['passenger_count'])

fare_by_passengers = df_analysis.groupby('passenger_count').agg({
    'fare_amount' : ['mean', 'median', 'std'],
    'fare_per_mile' : ['mean', 'median', 'std'],
    'fare_per_mile_per_passenger' : ['mean', 'median'],
    'trip_distance' : 'mean'
}).round(2)

print(fare_by_passengers)
print("\n")

fig, ax = plt.subplots(1, 3, figsize=(16, 5))

avg_fare = df_analysis.groupby('passenger_count')['fare_per_mile'].mean()
avg_fare.plot(kind='bar', ax=ax[0])
ax[0].set_xlabel('Passenger count')
ax[0].set_ylabel('Average fare per mile')
ax[0].set_title('Average fare per mile by passenger count')

avg_total_per_passenger =
df_analysis.groupby('passenger_count')['fare_per_mile_per_passenger'].mean
()
avg_total_per_passenger.plot(kind='bar', ax=ax[1])
ax[1].set_xlabel('Passenger count')
ax[1].set_ylabel('Average total per passenger')
ax[1].set_title('Average total per passenger by passenger count')

df_analysis.boxplot(column='fare_per_mile', by='passenger_count',
ax=ax[2])
ax[2].set_xlabel('Passenger count')
ax[2].set_ylabel('Trip distance')
ax[2].set_title('Trip distance by passenger count')

plt.suptitle('')
plt.tight_layout()
plt.show()
```

Analysis:

The cost per mile per passenger goes down as the number of passengers in a trip increases. This means that shared trips are more cost-effective. For a single passenger, the average cost per mile was $16.75. For two passengers, it decreased to $9.56. For groups of 5 to 6 people, it was around $5 to $6 per mile per person. This shows that group rides offer much better value compared to solo rides. However, the average cost per mile (not per passenger) was similar for all group sizes (ranging from $8.44 to $13.26). This means that drivers can earn a similar amount per mile regardless of the number of passengers. The main takeaway for managing operations is that focusing more on group rides doesn't mean drivers make less money, and it helps customers save money.

3.2.10. Find the average fare per mile by hours of the day and by days of the week

```
# Compare the average fare per mile for different days and for different
times of the day
df_analysis = df[(df['trip_distance'] > 0) &
                 (df['fare_amount'] > 0)].copy()

def distance_tier(distance):
  if distance <=2:
    return '0-2 miles'
  elif distance <=5:
    return '2-5 miles'
  else:
    return '5+ miles'

df_analysis['distance_tier'] =
df_analysis['trip_distance'].apply(distance_tier)
vendor_names = {1: 'Green Cab', 2: 'Yellow Cab'}
df_analysis['vendor_name'] = df_analysis['VendorID'].map(vendor_names)

tier_vendor_analysis = df_analysis.groupby(['distance_tier',
'vendor_name']).agg({
    'fare_amount': ['count', 'mean', 'median'],
    'trip_distance': 'mean',
    'total_amount': 'mean',
    'passenger_count': 'mean'
}).round(2)

print(tier_vendor_analysis)
print("\n")
```

```
tier_vendor_fare = df_analysis.pivot_table(
    values='fare_amount',
    index='distance_tier',
    columns='vendor_name',
    aggfunc='mean'
)
teir_vendor_fare = tier_vendor_fare.reindex(['0-2 miles', '2-5 miles', '5+
miles'])

tier_vendor_count = df_analysis.pivot_table(
    values='fare_amount',
    index='distance_tier',
    columns='vendor_name',
    aggfunc='count'
)
teir_vendor_count = tier_vendor_count.reindex(['0-2 miles', '2-5 miles',
'5+ miles'])
print("\nVendor Fare: ")
print(tier_vendor_fare.round(2))
print("\nVendor Count: ")
print(teir_vendor_count.astype(int))

fig, ax = plt.subplots(2, 2, figsize=(16, 10))

tier_vendor_fare.plot(kind='bar', ax=ax[0,0])
ax[0,0].set_xlabel('Distance tier')
ax[0,0].set_ylabel('Fare amount')
ax[0,0].set_title('Fare amount by distance tier and vendor')
plt.setp(ax[0,0].xaxis.get_majorticklabels(), rotation=45, ha='right')

tier_vendor_count.plot(kind='bar', ax=ax[0,1])
ax[0,1].set_xlabel('Distance tier')
ax[0,1].set_ylabel('Count')
ax[0,1].set_title('Count by distance tier and vendor')
ax[0,1].legend(title='Vendor')
plt.setp(ax[0,1].xaxis.get_majorticklabels(), rotation=45, ha='right')

tier_list = ['0-2 miles', '2-5 miles', '5+ miles']
positions = []
labels = []
data_plot=[]

for i, tier in enumerate(tier_list):
```

```
  for j, vendor in enumerate(['Green Cab', 'Yellow Cab']):
    data=df_analysis[(df_analysis['distance_tier'] == tier) &
(df_analysis['vendor_name'] == vendor)]['fare_amount']
    data_plot.append(data)
    positions.append(i*3 + j)
    labels.append(f"{tier}\n{vendor}")

ax[1,0].boxplot(data_plot, positions=positions, widths=0.6)
ax[1,0].set_title('Fare amount by distance tier and vendor')
ax[1,0].set_xlabel('Distance tier')
ax[1,0].set_ylabel('Fare amount')
ax[1,0].set_xticklabels(labels, fontsize=8)

stats_data=[]
for tier in tier_list:
  for vendor in ['Green Cab', 'Yellow Cab']:
    subset=df_analysis[(df_analysis['distance_tier'] == tier) &
(df_analysis['vendor_name'] == vendor)]
    stats_data.append({
        'Tier':tier,
        "Vendor":vendor,
        "Avg fare": subset['fare_amount'].mean(),
        "Fare per mile":(subset['fare_amount'] /
subset['trip_distance']).mean()
    })

stats_df = pd.DataFrame(stats_data)
pivot_stats = stats_df.pivot_table(index='Tier', columns='Vendor',
values='Fare per mile')
pivot_stats = pivot_stats.reindex(tier_list)

pivot_stats.plot(kind='bar', ax=ax[1,1])
ax[1,1].set_xlabel('Distance tier')
ax[1,1].set_ylabel('Fare per mile')
ax[1,1].set_title('Fare per mile by distance tier and vendor')
ax[1,1].legend(title='Vendor')
plt.setp(ax[1,1].xaxis.get_majorticklabels(), rotation=45, ha='right')

plt.tight_layout()
plt.show()
```

Analysis:

The average cost per mile peaked in the middle of the day (10 AM to 2 PM) at about $8.70 to $8.84 for Vendor 1, while in the early morning (0-6 AM), it was lower, around $6.68 to $7.18. This higher cost during midday is probably because of higher base fares for short city trips rather than longer highway trips. The cost per mile was pretty much the same every day of the week (around $7 to $9 per mile), with no big difference between weekdays and weekends. Looking at distances, short trips (0-2 miles) had a much higher cost per mile, which affects the average hourly rate during busy midday times when many short city trips are made.

3.2.11. Analyse the average fare per mile for the different vendor

```
# Compare fare per mile for different vendors
df_vendor = df[(df['trip_distance'] > 0) & (df['fare_amount'] > 0)].copy()
df_vendor['fare_per_mile'] = df_vendor['fare_amount'] /
df_vendor['trip_distance']
df_vendor['hour'] =
pd.to_datetime(df_vendor['tpep_pickup_datetime']).dt.hour

vendor_map = {1: 'Vendor 1 (CMT)', 2: 'Vendor 2 (VTS)'}
df_vendor['vendor_name'] =
df_vendor['VendorID'].map(vendor_map).fillna('Unknown')

vendor_hour_fare = df_vendor.groupby(['vendor_name',
'hour'])['fare_per_mile'].mean().round(2)
vendor_hour_fare = vendor_hour_fare.unstack(level=0)

print(vendor_hour_fare.to_string())
print("\n")

for vendor in vendor_hour_fare.columns:
  col=vendor_hour_fare[vendor].dropna()
  peak_hr=col.idxmax()
  low_hr=col.idxmin()
  print(f"Vendor: {vendor}")
  print(f"Overall avg fare: ${col.mean():.2f}")
  print(f"Peak hour: {peak_hr}:00")
  print(f"Low hour: {low_hr}:00")

fig, ax = plt.subplots(1, 2, figsize=(18, 6))

vendor_colors = {'Vendor 1 (CMT)': 'blue', 'Vendor 2 (VTS)': 'orange'}
for vendor in vendor_hour_fare.columns:
  ax[0].plot(
      vendor_hour_fare.index,
```

```
      vendor_hour_fare[vendor],
      marker='o',
      label=vendor
    )
ax[0].set_title('Avg fare per mole by vendor & hour')
ax[0].set_ylabel('Fare per mile')
ax[0].set_xlabel('Hour of the day')
ax[0].set_xticks(range(0, 24))
ax[0].legend(title = vendor)
ax[0].grid(True, alpha=0.3)

hour = vendor_hour_fare.index
x = np.arange(len(hour))
num_vendors = len(vendor_hour_fare.columns)
bar_width = 0.8 / num_vendors

for offset, (vendor, color) in enumerate(vendor_colors.items()):
  if vendor in vendor_hour_fare.columns:
    positions = x + offset * bar_width - (num_vendors-1) * bar_width/2
    ax[1].bar(positions, vendor_hour_fare[vendor], width=bar_width,
label=vendor, color=color, alpha=0.85)

ax[1].set_title('Avg fare per mile by vendor & hour')
ax[1].set_ylabel('Fare per mile')
ax[1].set_xlabel('Hour of the day')
ax[1].set_xticks(list(x))
ax[1].set_xticklabels([str(h) for h in hour])
ax[1].legend(title='Vendor')
ax[1].grid(True, alpha=0.3)

plt.suptitle('Average fare per mile', y=1.01)
plt.tight_layout()
plt.show()
```

Analysis:

Vendor 2 (VeriFone Inc / VTS) had a higher average cost per mile than Vendor 1 (Creative Mobile Technologies / CMT) throughout the day. Vendor 2 charged between $11.63 and $19.84 per mile, while Vendor 1 charged only $6.68 to $8.84 per mile. The biggest difference was during the early morning hours (4-5 AM), where Vendor 2 charged up to $19.84 per mile compared to $6.95 for Vendor 1. For both vendors, the cost per mile was highest during late night and early morning hours and lowest during the quiet pre-dawn period. Vendor 1's peak hours were 11 AM to 1 PM (~$8.70-8.84 per mile), while Vendor 2’s peak hours were 3-5 AM and 4-5 PM (up to $19.84 per mile). This suggests that the two vendors have different pricing

structures or types of trips.

3.2.12. Compare the fare rates of different vendors in a distance-tiered fashion

```
# Defining distance tiers
df_tier = df[(df['trip_distance'] > 0) & (df['fare_amount'] >0)].copy()
df_tier['fare_per_mile'] = df_tier['fare_amount'] /
df_tier['trip_distance']

vendor_map = {1: 'Vendor 1 (CMT)', 2: 'Vendor 2 (VTS)'}
df_tier['vendor_name'] =
df_tier['VendorID'].map(vendor_map).fillna('Unknown')

def distance_tier(distance):
  if distance <=2:
    return '0-2 miles'
  elif distance <=5:
    return '2-5 miles'
  else:
    return '5+ miles'

df_tier['distance_tier'] = df_tier['trip_distance'].apply(distance_tier)
tier_order = ['0-2 miles', '2-5 miles', '5+ miles']

pivot_avg = df_tier.pivot_table(
    values='fare_per_mile',
    index='distance_tier',
    columns='vendor_name',
    aggfunc='mean'
).round(2).reindex(tier_order)

pivot_count = df_tier.pivot_table(
    values='fare_per_mile',
    index='distance_tier',
    columns='vendor_name',
    aggfunc='count'
).reindex(tier_order).fillna(0).astype(int)

print(pivot_avg.to_string())
print("\n")
print(pivot_count.to_string())
```

```
for tier in tier_order:
  print(f"Tier: {tier}")
  for vendor in pivot_avg.columns:
    val = pivot_avg.loc[tier, vendor]
    cnt = int(pivot_count.loc[tier, vendor])
    print(f"{vendor}: ${val:.2f} ({cnt} trips)")
  print("\n")

vendor_colors = {'Vendor 1 (CMT)': 'blue', 'Vendor 2 (VTS)': 'orange'}
fig, ax = plt.subplots(1, 3, figsize=(18, 6))

pivot_avg.plot(kind='bar', ax=ax[0],
               color=[vendor_colors.get(c, 'grey') for c in
pivot_avg.columns] ,
               alpha=0.85)
ax[0].set_xlabel('Distance tier')
ax[0].set_ylabel('Fare per mile')
ax[0].set_title('Fare per mile by distance tier and vendor')
ax[0].set_xticklabels(tier_order, rotation=15, ha='right')
ax[0].legend(title='Vendor')
for container in ax[0].containers:
  ax[0].bar_label(container, fmt='$%.2f')

for vendor in pivot_avg.columns:
  ax[1].plot(range(len(tier_order)), pivot_avg[vendor].values,marker='o',
             label=vendor, linewidth=2.5, color=vendor_colors.get(vendor,
'grey'))
  for j, val in enumerate(pivot_avg[vendor].values):
    ax[1].annotate(f"${val:.2f}", (j, val), textcoords="offset points",
xytext=(0, 9),
                   ha='center')

ax[1].set_title('Fare per mile by distance tier and vendor')
ax[1].set_ylabel('Fare per mile')
ax[1].set_xlabel('Distance tier')
ax[1].set_xticks(range(len(tier_order)))
ax[1].set_xticklabels(tier_order, rotation=15, ha='right')
ax[1].legend(title='Vendor')
ax[1].grid(True, alpha=0.3)

pivot_count.plot(kind='bar', stacked=True, ax=ax[2],
```

```
                color=[vendor_colors.get(c, 'grey') for c in
pivot_avg.columns] ,
                alpha=0.85)
ax[2].set_xlabel('Distance tier')
ax[2].set_ylabel('Count')
ax[2].set_title('Count by distance tier and vendor')
ax[2].set_xticklabels(tier_order, rotation=15, ha='right')
ax[2].legend(title='Vendor')

plt.suptitle('Average fare per mile', y=1.01)
plt.tight_layout()
plt.show()
```

Analysis:

Looking at trips by distance, there was a big difference in pricing between the two vendors for short trips. For trips of 0-2 miles, Vendor 2 (VTS) charged an average of $18.10 per mile, compared to $9.89 per mile for Vendor 1 (CMT)—a 83% higher rate for Vendor 2. For medium trips (2-5 miles), the difference was much smaller: $6.55 per mile (VTS) versus $6.38 per mile (CMT), nearly the same. For longer trips (5+ miles), both vendors had similar rates of around $4.43 to $4.50 per mile. This pattern shows that Vendor 2 applies higher base or flag-fall fees that are more noticeable for short trips, while per-mile rates become more competitive for longer trips. The volume of trips also showed that Vendor 2 handles about three times more trips than Vendor 1 across all distance tiers, indicating that it is the main platform in terms of volume.

3.2.13. Analyse the tip percentages

```
#  Analyze tip percentages based on distances, passenger counts and pickup
times

df_tip = df[(df['payment_type'] == 1) &
            (df['fare_amount'] > 0) &
            (df['total_amount'] > 0) &
            (df['tip_amount'] > 0)].copy()

df_tip['tip_percentage'] = (df_tip['tip_amount'] / df_tip['total_amount'])
* 100
df_tip['hour'] = pd.to_datetime(df_tip['tpep_pickup_datetime']).dt.hour
df_tip['day_week'] =
pd.to_datetime(df_tip['tpep_pickup_datetime']).dt.day_name()

def dist_tier(d):
```

```
  if d<=2: return '0-2 miles'
  elif d<=5: return '2-5 miles'
  else: return '5+ miles'

df_tip['distance_tier'] = df_tip['trip_distance'].apply(dist_tier)
tier_order = ['0-2 miles', '2-5 miles', '5+ miles']

df_tip['pax_group'] = df_tip['passenger_count'].apply(
    lambda x: '1 passenger' if x==1
    else ('2 passengers' if x==2
    else ('3-4 passengers' if x in [3-4]
    else '5+ passengers'))
)
pax_order = ['1 passenger', '2 passengers', '3-4 passengers', '5+
passengers']

tip_by_dist =
df_tip.groupby('distance_tier')['tip_percentage'].agg(['mean', 'median',
'count']).round(2)
tip_by_dist = tip_by_dist.reindex(tier_order)
print(tip_by_dist.rename(columns={'mean': 'Avg Tip%', 'median': 'Median
Tip%', 'count': 'Trips'}).to_string())
print("\n")

tip_by_pax = df_tip.groupby('pax_group')['tip_percentage'].agg(['mean',
'median', 'count']).round(2)
tip_by_pax = tip_by_pax.reindex(pax_order)
print(tip_by_pax.rename(columns={'mean': 'Avg Tip%', 'median': 'Median
Tip%', 'count': 'Trips'}).to_string())
print("\n")

tip_by_hour = df_tip.groupby('hour')['tip_percentage'].mean().round(2)
print(tip_by_hour.to_string())
print("\n")

day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday',
'Saturday', 'Sunday']
tip_by_day =
df_tip.groupby('day_week')['tip_percentage'].mean().round(2).reindex(day_o
rder)
print(tip_by_day.to_string())
print("\n")
```

```
low_tip_threshold = df_tip['tip_percentage'].quantile(0.25)
df_low = df_tip[df_tip['tip_percentage'] < low_tip_threshold]
df_high = df_tip[df_tip['tip_percentage'] >= low_tip_threshold]
print(f'Avg distance  - Low tip: {df_low["trip_distance"].mean():.2f} mi |
High tip: {df_high["trip_distance"].mean():.2f} mi')
print(f'Avg passengers- Low tip:
{df_low["passenger_count"].mean():.2f}   | High tip:
{df_high["passenger_count"].mean():.2f}')
print(f'Avg hour      - Low tip: {df_low["hour"].mean():.1f}h       | High
tip: {df_high["hour"].mean():.1f}h')

fig, ax = plt.subplots(2, 2, figsize=(18,12))

bars1 = ax[0,0].bar(tip_by_dist.index, tip_by_dist['mean'],
                    alpha=0.85)
ax[0,0].set_xlabel('Distance tier')
ax[0,0].set_ylabel('Avg tip %')
ax[0,0].set_title('Average tip % by trip distance')
ax[0,0].bar_label(bars1, fmt='%.1f%%')

valid_pax = [p for p in pax_order if p in tip_by_pax.index]
bars2 = ax[0,1].bar(valid_pax, tip_by_pax.loc[valid_pax, 'mean'],
                    alpha=0.85)
ax[0,1].set_title('Avg tip % by passenger count')
ax[0,1].set_xlabel('Passenger count')
ax[0,1].set_ylabel('Avg tip %')
ax[0,0].bar_label(bars1, fmt='%.1f%%')

ax[1,0].plot(tip_by_hour.index, tip_by_hour.values,
             marker='o', linewidth=2)
ax[1,0].fill_between(tip_by_hour.index, tip_by_hour.values, alpha=0.15)
ax[1,0].set_title('Avg tip % by pickup hour')
ax[1,0].set_xlabel('Hour of the day')
ax[1,0].set_ylabel('Avg tip %')
ax[1,0].grid(True, alpha=0.3)
ax[1,0].set_xticks(range(0, 24))
ax[1,0].set_xticklabels([str(h) for h in range(0, 24)])

low_hr = tip_by_hour.idxmin()
ax[1,0].axvline(x=low_hr, color='red', linestyle='--', label=f'Lowest:
{low_hr}:00')
ax[1,0].legend()
```

```
bars4 = ax[1,1].bar(day_order, tip_by_day.values)
ax[1,1].set_title('Avg tip % by day of the week')
ax[1,1].set_xlabel('Day of the week')
ax[1,1].set_ylabel('Avg tip %')
ax[1,1].bar_label(bars4, fmt='%.1f%%')
ax[1,1].set_xticklabels(day_order, rotation=30, ha='right')

from matplotlib.patches import Patch
ax[1,1].legend(handles=[
    Patch(facecolor='blue', label='Weekday'),
    Patch(facecolor='green', label='Weekend')])

plt.suptitle('Tip percentage analysis: Distance, Passengers and pickup
time')
plt.tight_layout()
plt.show()
```

Analysis:

Tip analysis was limited to paid trips using credit cards with positive tips, covering 1,503,802 trips. The average tip was about 15.4% of the total fare. By distance, short trips (0-2 miles) had an average tip of 15.65%, medium trips (2-5 miles) had 15.17%, and long trips (5+ miles) had 15.12%—showing a slight but consistent decrease in tipping for longer trips. By passenger group, the tip percentage was almost the same for 1-passenger (15.42%), 2-passenger (15.44%), and 5+ passenger (15.48%) trips, meaning the number of passengers didn't greatly affect tipping. By hour, midday hours (10 AM to 1 PM) had slightly higher tips (~15.68-15.72%) while night hours (midnight to 5 AM) had slightly lower tips (~15.20-15.23%). By day of the week, tip percentages were almost the same (15.40-15.46%), suggesting that the tipping culture among credit card users in NYC taxis is very consistent.

3.2.14. Analyse the trends in passenger count

```
# See how passenger count varies across hours and days
day_names = {0: 'Monday', 1:'Tuesday', 2:'Wednesday', 3:'Thursday',
4:'Friday', 5:'Saturday', 6:'Sunday'}
df_pax = df.copy()

df_pax['day_week'] =
pd.to_datetime(df_pax['tpep_pickup_datetime']).dt.dayofweek
df_pax['day_name'] = df_pax['day_week'].map(day_names)
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday',
'Saturday', 'Sunday']
```

```
pax_by_hour = df_pax.groupby('hour')['passenger_count'].mean().round(2)
pax_by_day =
df_pax.groupby('day_name')['passenger_count'].mean().reindex(day_order).ro
und(2)

heatmap_data = df_pax.groupby(['day_name',
'hour'])['passenger_count'].mean().unstack(level=1).reindex(day_order).rou
nd(2)
print("\n", pax_by_day.to_string())
print(f"\nBusiest day: {pax_by_day.idxmax()} → avg {pax_by_day.max():.2f}
passengers")
print(f"Quietest day: {pax_by_day.idxmin()} → avg {pax_by_day.min():.2f}
passengers")

fig, ax = plt.subplots(2, 2, figsize=(18, 12))

ax[0,0].plot(pax_by_hour.index, pax_by_hour.values,
             marker='o', linewidth=2)
ax[0,0].fill_between(pax_by_hour.index, pax_by_hour.values, alpha=0.2)
ax[0,0].set_title('Avg passenger count by pickup hour')
ax[0,0].set_xlabel('Hour of the day')
ax[0,0].set_ylabel('Avg passenger count')
ax[0,0].set_xticks(range(0, 24, 2))
ax[0,0].grid(True, alpha=0.3)

colors_day = ['blue', 'green', 'orange', 'red', 'purple', 'brown', 'pink']
bars = ax[0,1].bar(day_order, pax_by_day.values, linewidth=1)
ax[0,1].set_title('Avg passenger count by day of the week')
ax[0,1].set_xlabel('Day of the week')
ax[0,1].set_ylabel('Avg passenger count')
ax[0,1].tick_params(axis='x', rotation=30)
for bar, val in zip(bars, pax_by_day.values):
  ax[0,1].text(bar.get_x() + bar.get_width() / 2, bar.get_height() +
0.005,
               f'{val:.2f}', ha='center', va='bottom')

sns.heatmap(heatmap_data, cmap='YlOrRd', ax=ax[1,0],
            cbar_kws={'label' : 'Avg Passengers'}, linewidth=0.3)
ax[1,0].set_title('Avg passenger count by day of the week & hour')
ax[1,0].set_xlabel('Hour of the day')
ax[1,0].set_ylabel('Day of week')
```

```
box_data = [df_pax[df_pax['day_name'] == d]['passenger_count'].values for
d in day_order]
bp = ax[1,1].boxplot(box_data, labels=day_order,
                     patch_artist=True, showfliers=False)
for patch, color in zip(bp['boxes'], colors_day):
  patch.set_facecolor(color); patch.set_alpha(0.7)
ax[1,1].set_title('Avg passenger count by day of the week')
ax[1,1].set_xlabel('Day of the week')
ax[1,1].set_ylabel('Avg passenger count')
ax[1,1].tick_params(axis='x', rotation=30)

plt.tight_layout()
plt.show()

print(f"• Hourly range: {pax_by_hour.min():.2f} – {pax_by_hour.max():.2f}
passengers")
print(f"• Late-night hours (0–4 AM) tend to have higher avg passengers "
      f"(likely group rides/nightlife)")
print(f"• Weekends ({pax_by_day['Saturday']:.2f},
{pax_by_day['Sunday']:.2f} avg) "
      f"vs Weekdays
({pax_by_day[['Monday','Tuesday','Wednesday','Thursday','Friday']].mean():
.2f} avg)")
print("• Passenger count shows relatively modest variation overall, "
      "suggesting most trips are solo regardless of time/day")
```

Analysis:

Most NYC taxi rides (76.5%) are single-passenger trips. Two-passenger trips made up 15.3%, three-passenger trips 3.8%, four-passenger trips 2.2%, and five or six passengers combined under 2%. On average, the number of passengers per trip ranged narrowly from 1.26 to 1.45. Late-night hours (midnight to 4 AM) had slightly higher average passenger counts (~1.40-1.45), which is consistent with group travel for nightlife. By day of the week, Saturday had the highest average passenger count (1.49) and Tuesday the lowest (1.34), showing more social or group travel on weekends. Passenger counts remained stable across months, indicating there is no big change in group riding behavior due to seasons.

3.2.15. Analyse the variation of passenger counts across zone

```
# How does passenger count vary across zones

passenger_by_zone = (
    df_zones.groupby('pu_zone')['passenger_count']
    .agg(['mean', 'median', 'std', 'count'])
```

```
    .round(2)
    .rename(columns={
        'mean': 'Avg_passengers',
        'median': 'Median_passengers',
        'std': 'Std_dev',
        'count': 'Trip_count'
    })
    .sort_values('Avg_passengers', ascending=False)
)

passenger_by_zone = passenger_by_zone[passenger_by_zone['Trip_count'] >=
50]
print(f"\nTotal zones analysed (≥50 trips): {len(passenger_by_zone)}")
print(f"\nTop 10 Zones — Highest Average Passenger Count:")
print(passenger_by_zone.head(10).to_string())
print(f"\nBottom 10 Zones — Lowest Average Passenger Count:")
print(passenger_by_zone.tail(10).to_string())
print(f"\nOverall Statistics:")
print(f"  Global avg passengers :
{df_with_zones['passenger_count'].mean():.2f}")
print(f"  Highest zone avg      :
{passenger_by_zone['Avg_passengers'].max():.2f}  →
{passenger_by_zone['Avg_passengers'].idxmax()}")
print(f"  Lowest zone avg       :
{passenger_by_zone['Avg_passengers'].min():.2f}  →
{passenger_by_zone['Avg_passengers'].idxmin()}")
print(f"  Avg std dev per zone  :
{passenger_by_zone['Std_dev'].mean():.2f}")

fig, ax = plt.subplots(2, 2, figsize=(18, 12))

top15=passenger_by_zone.head(15)
ax[0,0].barh(top15.index[::-1], top15['Avg_passengers'][::-1],
             alpha=0.8)
ax[0,0].set_title('Avg passenger count by zone')
ax[0,0].set_xlabel('Avg passenger count per trip')
for i, (zone, row) in enumerate(top15[::-1].iterrows()):
  ax[0,0].text(row['Avg_passengers'] + 0.01, i,
               f"{row['Avg_passengers']:.2f}", va='center')

bot15 = passenger_by_zone.tail(15)
ax[0,1].barh(bot15.index[::-1], bot15['Avg_passengers'][::-1],
             alpha=0.8)
```

```
ax[0,1].set_title('Bottom 15 zones Avg passengers')
ax[0,1].set_xlabel('Average passengers per trip')
for i, (zone, row) in enumerate(bot15[::-1].iterrows()):
  ax[0,1].text(row['Avg_passengers'] + 0.01, i,
               f"{row['Avg_passengers']:.2f}", va='center')

ax[1,0].hist(passenger_by_zone['Avg_passengers'], bins=30, alpha=0.75)
ax[1,0].axvline(passenger_by_zone['Avg_passengers'].mean(), color='red',
linestyle='--',
                linewidth=2, label=f"Mean:
{passenger_by_zone['Avg_passengers'].mean():.2f}")
ax[1,0].set_title('Avg passenger count by zone')
ax[1,0].set_xlabel('Avg passenger count per trip')
ax[1,0].set_ylabel('Count of zones')
ax[1,0].legend()
ax[1,0].grid(alpha=0.3)

pax_by_borough=(
    df_with_zones.groupby('pu_borough')['passenger_count']
    .mean()
    .round(2)
    .sort_values(ascending=False)
)
bars = ax[1,1].bar(pax_by_borough.index, pax_by_borough.values, alpha=0.8)
ax[1,1].set_title('Avg passenger count by borough')
ax[1,1].set_xlabel('Borough')
ax[1,1].set_ylabel('Avg passenger count per trip')
ax[1,1].tick_params(axis='x', rotation=30)
for bar, val in zip(bars, pax_by_borough.values):
  ax[1,1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
                                          f'{val:.2f}', ha='center')

plt.tight_layout()
plt.show()
```

Analysis:

Among 184 zones with at least 50 trips, the average number of passengers per trip ranged from 1.00 to 2.00. The top zones by average passenger count were: Rikers Island (2.00), Oakwood (2.00), Arrochar/Fort Wadsworth (1.97), Red Hook (1.85), and Newark Airport (1.84), Battery Park (1.82). These zones are mostly destination points or transit hubs where group travel is more common. At the borough level, EWR (1.84), Manhattan (1.39), Queens (1.21), Brooklyn (1.21), Staten Island (1.14), and Bronx (1.11) had the highest averages. The outer borough

zones had both the highest individual averages and the most variation, reflecting a wide range of trip types. Manhattan zones had more consistent, almost solo averages due to the dominance of work and commute trips.

3.2.16. Analyse the pickup/dropoff zones or times when extra charges are applied more frequently.

```
# For a more detailed analysis, we can use the zones_with_trips
GeoDataFrame
# Create a new column for the average passenger count in each zone.

avg_pax_per_zone = (
    df_zones.groupby('PULocationID')['passenger_count']
    .mean()
    .round(3)
    .reset_index()
    .rename(columns={'passenger_count': 'avg_passenger_count',
                     'PULocationID': 'LocationID'})
)

zone_with_trips = zones.merge(avg_pax_per_zone, on='LocationID',
how='left')
zone_with_trips['avg_passenger_count'] =
zone_with_trips['avg_passenger_count'].fillna(0)
print(f"\n Shape: {zone_with_trips.shape}")
print(
    zone_with_trips[['zone', 'borough', 'avg_passenger_count']]
    .sort_values('avg_passenger_count', ascending=False)
    .head(10)
    .to_string(index=False)
)
print(f"\nZones with no trip data (avg = 0): "
      f"{(zone_with_trips['avg_passenger_count'] == 0).sum()}")

fig, ax = plt.subplots(1,2, figsize=(20, 10))
fig.suptitle('Geo Analysis')

zone_with_trips.plot(ax=ax[0], column='avg_passenger_count',
cmap='YlOrRd', legend=True,
                     legend_kwds={'label':'Avg passenger count'},
missing_kwds={'color':'lightgrey', 'label':'No Data'})
ax[0].set_title('Avg passenger count by zone')
ax[0].axis('off')
```

```
threshold = zone_with_trips['avg_passenger_count'].quantile(0.75)
zone_with_trips['high_pax'] = zone_with_trips['avg_passenger_count'] >=
threshold

zone_with_trips.plot(ax=ax[1], color='lightgrey', linewidth=0.3)
zone_with_trips[zone_with_trips['high_pax']].plot(
    ax=ax[1],
    column='avg_passenger_count',
    cmap='Reds',
    legend=True,
    linewidth=0.5,
    legend_kwds={'label':'Avg passenger count', 'shrink':0.6}
)
ax[1].set_title('Zones with high avg passenger count')
ax[1].axis('off')
plt.tight_layout()
plt.show()

print("\n Borough- wise summary")
borough_summary = (
    zone_with_trips[zone_with_trips['avg_passenger_count'] > 0]
    .groupby('borough')['avg_passenger_count']
    .agg(['mean', 'max', 'min', 'count'])
    .round(2)
    .rename(columns={
        'mean': 'Avg_passengers',
        'max': 'Max_zone_avg',
        'min': 'Min_zone_avg',
        'count': 'Zone_count'
    }).sort_values('Avg_passengers', ascending=False)
)

print(borough_summary.to_string())
top_zone = zone_with_trips.sort_values('avg_passenger_count',
ascending=False).iloc[0]
print(f"• Zone with highest avg passengers : {top_zone['zone']} "
      f"({top_zone['borough']}) → {top_zone['avg_passenger_count']:.2f}")
print(f"• Top-quartile threshold            : ≥ {threshold:.2f} avg
passengers")
print(f"• Airport zones (JFK/LGA) cluster in high-passenger regions —
group travel effect")
```

```
print(f"• Outer borough zones show wider variation than central Manhattan
zones")
print(f"• Zones with no data (inactive/unused): "
      f"{(zone_with_trips['avg_passenger_count'] == 0).sum()}")
```

Analysis:

Surcharge prevalence across 1,354,12 trips: Improvement surcharge (100.0% of trips, avg $1.00), MTA tax (99.1%, avg $0.50), Congestion surcharge (92.3%, avg $2.50), Rush hour/Overnight extra (61.2%, avg $2.63), Airport fee (8.9%, avg $1.62), and Tolls (8.2%, avg $7.34). By time: The Rush/Overnight extra peaks during late-night hours (midnight-1 AM) and evening rush (4-8 PM). Congestion surcharge is concentrated in daytime business hours (8 AM to 6 PM). Airport fee is almost exclusively applied at JFK and LaGuardia pickup zones. Tolls cluster in zones near bridges and tunnels serving outer boroughs and airport routes. By zone: Upper East Side South had the highest any-extra-charge rate at 99.9%, while Arrochar/Fort Wadsworth had 100% surcharge application for drop-offs, reflecting its location near tolled infrastructure. Weekdays showed higher congestion surcharge rates than weekends.

4. Conclusions

4.1. Final Insights and Recommendations

4.1.1. Recommendations to optimize routing and dispatching based on demand patterns and operational inefficiencies.

```
trips_by_hour = df_zones.groupby('hour').size().reset_index(name='trips')
trips_by_hour['pct_of_day'] = (trips_by_hour['trips'] /
trips_by_hour['trips'].sum() * 100).round(2)

day_order = ['Monday', 'Tuesday' , 'Wednesday' , 'Thursday' , 'Friday' ,
'Saturday' , 'Sunday']
trips_by_day =
df_zones.groupby('day_name').size().reindex(day_order).reset_index(name='t
rips')

heatmap_data = (
       df_zones.groupby(['day_name','hour'])
       .size()
       .unstack(fill_value=0)
       .reindex(day_order)
)
```

```
df_zones['duration_min'] = (
pd.to_datetime(df_zones['tpep_dropoff_datetime']) -
pd.to_datetime(df_zones['tpep_pickup_datetime'])
).dt.total_seconds() / 60

df_speed = df_zones[(df_zones['duration_min'] > 0) &
(df_zones['trip_distance' ] > 0)].copy()
df_speed['speed_mph'] = (df_speed['trip_distance'] /
df_speed['duration_min' ]) * 60

speed_by_hour = df_speed.groupby('hour')['speed_mph'].mean().round(2)

top_pu_zones = (
    df_zones.groupby('pu_zone').size()
    .sort_values(ascending=False)
    .head(15)
    .reset_index(name='trip_count' )
)

peak_hour = trips_by_hour.loc[trips_by_hour['trips' ]. idxmax(), 'hour' ]
trough_hour = trips_by_hour.loc[trips_by_hour['trips'].idxmin(), 'hour']
peak_day = trips_by_day.loc[trips_by_day['trips'].idxmax(), 'day_name']

fig, axes = plt.subplots(2, 2, figsize=(18, 12))
fig.suptitle('4.1.1 ROUTING & DISPATCHING ANALYSIS', fontsize=14,
fontweight='bold')

colors_hour = ['#d62728' if h in range(17, 21) else
'#ff7f0e' if h in range(7, 10) else '#1f77b4'
for h in trips_by_hour['hour']]
axes[0,0].bar(trips_by_hour['hour'], trips_by_hour['trips'],
color=colors_hour, edgecolor='black', alpha=0.85)
axes[0,0].set_title('Trip Volume by Hour of Day', fontweight='bold')
axes[0,0].set_xlabel('Hour')
axes[0,0].set_ylabel('Number of Trips')
axes[0,0].set_xticks(range(0, 24,2))
axes[0,0].legend(handles=[
    plt.Rectangle((0,0),1,1, color='#d62728', label='Evening Peak (5-8
PM)'),
    plt.Rectangle((0,0),1,1, color='#ff7f0e', label='Morning Rush (7-10
AM)'),
    plt.Rectangle((0,0),1,1, color='#1f77b4', label='Off-Peak')
```

```
])
axes[0,0].grid(axis='y', alpha=0.3)

axes[0,1].plot(speed_by_hour.index, speed_by_hour.values,
marker='o', color='darkgreen', linewidth=2.5)
axes[0,1].fill_between(speed_by_hour.index, speed_by_hour.values,
alpha=0.2, color='green')
axes[0,1].axhline(10, color='red', linestyle='--', linewidth=1.5,
label='10 mph congestion threshold')
axes[0,1].set_title('Avg Trip Speed by Hour (Congestion Indicator)',
fontweight='bold')
axes[0,1].set_xlabel('Hour')
axes[0,1].set_ylabel('Speed (mph)')
axes[0,1].set_xticks(range(0,24,2))
axes[0,1].legend()
axes[0,1].grid(alpha=0.3)

im = axes[1,0].imshow(heatmap_data.values, cmap='YlOrRd', aspect='auto')
axes[1,0].set_xticks(range(0,24,2))
axes[1,0].set_xticklabels(range(0,24,2))
axes[1,0].set_yticks(range(len(day_order)))
axes[1,0].set_yticklabels(day_order)
axes[1,0].set_title('Demand Heatmap: Hour x Day of Week',
fontweight='bold')
axes[1,0].set_xlabel('Hour of Day')
plt.colorbar(im, ax=axes[1,0], label='Trip Coup')

axes[1,1].barh(top_pu_zones['pu_zone'][::-1], top_pu_zones['trip_count' ][
: :- 1],
color='steelblue', edgecolor='black', alpha=0.85)
axes[1,1].set_title('Top 15 Pickup Zones by Volume', fontweight='bold')
axes[1,1].set_xlabel('Number of Trips')
axes[1,1].grid(axis='x', alpha=0.3)

plt.tight_layout()
plt.show()
```

Analysis:

1. Shift supply to match peak demand windows: The evening peak (3-8 PM, especially 5-7 PM) accounts for a disproportionate share of daily trips (~35-40% of daily volume). Dispatching should maximise cab availability during these hours, particularly in Midtown

Manhattan, Upper East Side, and Penn Station zones where demand spikes sharply. Conversely, reducing idle fleet size between 2-5 AM when demand is minimal (under 5% of daily volume) reduces operating costs.

2. Avoid congested intra-Midtown routes during midday: Average speeds on short Midtown routes at 11 AM-3 PM drop to 3-7 mph. Dispatchers should route cabs via alternate corridors (e.g., 2nd/3rd Avenue instead of 5th/6th during midday) and use real-time traffic signals to dynamically re-route.

3. Rebalance cabs after airport drops: Zones like Bayside, Ridgewood, and Greenpoint are heavy drop-off destinations with very low pickup ratios (0.05-0.06). After delivering passengers to these zones, drivers should be directed immediately toward high-demand pickup hubs rather than waiting for organic demand.

4. Incentivise early-morning operations near nightlife zones: Lower East Side (51.7% night trips), East Village (35.4%), and West Village (30.7%) maintain strong demand through the night. A dedicated late-night fleet assignment to these zones would capture demand that currently goes unmet.

5. Monitor and flag outlier slow routes: The 36,685 identified route-hour pairs with average speed below 10 mph represent operational bottlenecks. Building a real-time alert system for these combinations enables proactive rerouting.

4.1.2. Suggestions on strategically positioning cabs across different zones to make best use of insights uncovered by analysing trip trends across time, days and months.

```
zone_stats = (
    df_zones.groupby('pu_zone')
    .agg(
        trip_count = ('trip_distance', 'count' ),
        avg_fare = ('fare_amount', 'mean' ),
        avg_distance = ('trip_distance', 'mean' ),
        avg_passengers = ('passenger_count' , 'mean' ),
        peak_hour = ('hour', lambda x: x.mode() [0])
    ).round(2)
     .sort_values('trip_count', ascending=False)
)

df_zones['month'] =
pd.to_datetime(df_zones['tpep_pickup_datetime']).dt.month
month_names = {1:'Jan' ,2: 'Feb' , 3: 'Mar' , 4: 'Apr' , 5: 'May' , 6:
'Jun',
7:'Jul' , 8: 'Aug' , 9: 'Sep' , 10: 'Oct' , 11: 'Nov', 12: 'Dec'}
```

```
df_zones['month_name'] = df_zones['month'].map(month_names)
month_order = list(month_names. values())

trips_by_month = (
    df_zones.groupby('month')
    .size()
    .reset_index(name='trip_count')
)
trips_by_month['month_name']= trips_by_month['month'].map(month_names)

df_zones['time_period' ] = df_zones['hour'].apply(
        lambda h: 'Night (10PM-5AM)' if h >= 22 or h <= 5
        else ('Morning Rush (6-10AM)' if 6 <= h <= 10
        else ('Evening Peak (5-9PM)' if 17 <= h <= 21
        else 'Daytime (11AM-4PM)'))
)

top_zones_by_period = (
    df_zones.groupby(['time_period', 'pu_zone'])
    .size()
    .reset_index(name='trips')
    .sort_values(['time_period' , 'trips'], ascending=[True, False])
    .groupby('time_period' )
    .head(5)
)

tier1 = zone_stats.head(5).index.tolist()
tier2 = zone_stats.iloc[5:15].index.tolist()

print(top_zones_by_period.to_string(index=False))

fig, axes = plt.subplots(2, 2, figsize=(18,12))
fig.suptitle('STRATEGIC CAB POSITIONING', fontsize=14, fontweight='bold')

top20 = zone_stats.head(20)
tier_colors = ['#d62728' ]*5 + ['#ff7f0e']*10 + ['#2ca02c']*5
axes[0,0].barh(top20.index[::-1], top20['trip_count' ][::-1],
color=tier_colors[ : : -1], edgecolor='black', alpha=0.85)
axes[0,0].set_title('Top 20 Pickup Zones by Trip Volume',
fontweight='bold')
axes[0,0].set_xlabel('Number of Trips')
axes[0,0].grid(axis='x', alpha=0.3)
```

```
axes[0,1].plot(trips_by_month['month'], trips_by_month['trip_count' ],
marker='o', linewidth=2.5, color='steelblue', markersize=8)
axes[0,1].fill_between(trips_by_month['month'],
trips_by_month['trip_count' ],
                       alpha=0.2, color='steelblue')
axes[0,1].set_title('Monthly Trip Volume - Seasonality',
fontweight='bold')
axes[0,1].set_xlabel('Month')
axes[0,1].set_ylabel('Number of Trips')
axes[0,1].set_xticks(range(1,13))
axes[0,1].set_xticklabels(month_order, rotation=30)
axes[0,1].grid(alpha=0.3)

top_fare = zone_stats.nlargest(15, 'avg_fare')
axes[1,0].barh(top_fare.index[::-1], top_fare['avg_fare'][::-1],
color='mediumpurple', edgecolor='black', alpha=0.85)
axes[1,0].set_title('Top 15 Zones by Average Fare', fontweight='bold')
axes[1,0].set_xlabel('Average Fare ($)')
for i, (zone, row) in enumerate(top_fare[::-1].iterrows()):
  axes[1,0].text(row['avg_fare'] +0.2, i, f"${row['avg_fare']:.2f}",
va='center')
  axes[1,0].grid(axis='x', alpha=0.3)

period_order = ['Morning Rush (6-10AM)','Daytime (11AM-4PM)',
'Evening Peak (5-9PM)','Night (10PM-5AM)']
period_counts = (
    df_zones.groupby('time_period' ).size()
    .reindex(period_order)
)

period_colors = ['#ff7f0e','#2ca02c','#d62728','#1f77b4']
bars = axes[1,1].bar(period_counts. index, period_counts.values,
                     color=period_colors, edgecolor='black', alpha=0.85)
axes[1,1].set_title('Trip Volume by Time Period', fontweight='bold')
axes[1,1].set_xlabel('Time Period')
axes[1,1].set_ylabel('Number of Trips')
axes[1,1].tick_params(axis='x', rotation=20)
for bar, val in zip(bars, period_counts.values):
  axes[1,1].text(bar.get_x()+ bar.get_width()/2, bar.get_height() + 100,
                 f'{val:,}', ha='center', fontsize=9, fontweight='bold')

plt.tight_layout()
plt.show()
```

Analysis:
Zone positioning strategy should be tiered by time period: During Daytime (11 AM-4 PM), concentrate supply in Upper East Side South, Upper East Side North, Midtown Center, and JFK Airport — the four highest-volume daytime zones accounting for over 130,000 trips during this window. During Evening Peak (5-9 PM), prioritise Midtown Center (33,691), JFK Airport (33,318), Upper East Side South (29,789), and Midtown East (24,930). During Morning Rush (6-10 AM), position cabs near Upper East Side North (18,488), Penn Station/Madison Sq West (15,801), JFK Airport (15,083), and Midtown East (12,652) to capture commuter flows from transit hubs. During Night (10 PM-5 AM), JFK Airport is the top zone (22,909 night trips), followed by nightlife zones in Lower East Side, East Village, and West Village. Monthly positioning: In summer (July-August), when Manhattan demand dips, shift some capacity toward tourist-heavy and airport zones. In spring and fall (March, May, October), maximise Manhattan-wide coverage as these months generate the highest revenue. Weekend-specific strategy: On Saturdays and Sundays, delay morning deployment by 1-2 hours (starting 9-10 AM rather than 7 AM) and extend coverage through 2 AM, with emphasis on nightlife zones in Lower East Side and Greenwich Village. Zones with high pickup-to-dropoff ratios (East Elmhurst, JFK, LaGuardia) should be treated as anchor zones where cabs return after completing outer-borough drop-offs, minimising deadhead miles.

4.1.3. Propose data-driven adjustments to the pricing strategy to maximize revenue while maintaining competitive rates with other vendors.

```
df_zones = df_zones.sample(frac=0.3, random_state=42)
df_pricing = df_zones[(df_zones['trip_distance'] > 0) &
(df_zones['fare_amount'] > 0)]
df_pricing['fare_per_mile'] = (df_pricing['fare_amount'] /
df_pricing['trip_distance']).round(2)

fare_by_hour = df_pricing.groupby('hour').agg(
    avg_fare = ('fare_amount', 'mean'),
    avg_fare_per_mi = ('fare_per_mile', 'mean'),
    trip_count = ('fare_amount', 'count')
).round(2)

def dist_tier(d):
  if d<=2 :return '0-2 miles (Short)'
  elif d <= 5: return '2-5 miles (Medium)'
```

```
  return '5+ miles (Long)'

df_pricing['dist_tier'] = np.select(
    [
        df_pricing['trip_distance'] <= 2,
        df_pricing['trip_distance'] <= 5
    ],
    [
        '0-2 miles (Short)',
        '2-5 miles (Medium)'
    ],
    default='5+ miles (Long)'
)
tier_order = ['0-2 miles (Short)', '2-5 miles (Medium)', '5+ miles
(Long)']

fare_by_tier = (
    df_pricing.groupby('dist_tier')
    .agg(avg_fare=('fare_amount','mean'),
    avg_fare_per_mi=('fare_per_mile', 'mean'),
    trip_count=('fare_amount','count'))
    .reindex(tier_order)
    .round(2)
)

df_tips = df_zones[(df_zones['payment_type'] == 1) &
(df_zones['fare_amount'] > 0)]
df_tips['tip_pct'] = (df_tips['tip_amount'] /
df_tips['fare_amount']*100).round(2)
avg_tip_pct = df_tips['tip_pct'].mean()
del df_tips

vendor_map = {1: 'Creative Mobile Tech', 2: 'VeriFone Inc'}
df_zones['vendor_name'] = df_zones['VendorID'].map(vendor_map)
vendor_stats = (
    df_zones.groupby('vendor_name')
    .agg(avg_fare=('fare_amount','mean'),
    trip_count=('fare_amount','count'),
    avg_distance=('trip_distance','mean'))
    .round(2)
)

def surge_multiplier(h):
```

```
  if h in range(17, 21): return 1.30
  elif h in range(7, 10): return 1.20
  elif h in range(22, 24) or h in range(0, 2): return 1.15
  elif h in range(2, 6):return 0.85
  else: return 1.00

fare_by_hour['surge_multiplier'] =
fare_by_hour.index.map(surge_multiplier)
fare_by_hour['proposed_fare'] = (fare_by_hour['avg_fare'] *
fare_by_hour['surge_multiplier']).round(2)
fare_by_hour['revenue_uplift'] = fare_by_hour['proposed_fare'] -
fare_by_hour['avg_fare']

fig, axes = plt.subplots(2, 2, figsize=(18,12))
fig.suptitle('4.1.3 DATA-DRIVEN PRICING STRATEGY', fontsize=14,
fontweight='bold')

axes[0,0].plot(fare_by_hour.index, fare_by_hour['avg_fare'],
         marker='o', linewidth=2.5, label='Current Avg Fare',
color='steelblue')
axes[0,0].plot(fare_by_hour.index, fare_by_hour['proposed_fare'],
         marker='s', linewidth=2.5, linestyle='--', label='Proposed Fare',
color='darkorange')
axes[0,0].fill_between(fare_by_hour.index,
fare_by_hour['avg_fare'], fare_by_hour['proposed_fare'],
alpha=0.2, color='green', label='Revenue Uplift')
axes[0,0].set_title('Current vs Proposed Fare by Hour', fontweight='bold')
axes[0,0].set_xlabel('Hour of Day')
axes[0,0].set_ylabel('Average Fare ($)')
axes[0,0].set_xticks(range(0,24,2))
axes[0,0].legend()
axes[0,0].grid(alpha=0.3)

mult_colors = ['#d62728' if m > 1.1 else '#ff7f0e' if m > 1.0
               else '#2ca02c' if m == 1.0 else '#1f77b4'
               for m in fare_by_hour['surge_multiplier']]
axes[0,1].bar(fare_by_hour.index, fare_by_hour['surge_multiplier'],
color=mult_colors, edgecolor='black', alpha=0.85)
axes[0,1].axhline(1.0, color='black', linestyle='--', linewidth=1.5,
label='Baseline 1.0x')
axes[0,1].set_title('Proposed Surge Multipliers by Hour',
fontweight='bold' )
axes[0,1].set_xlabel('Hour of Day')
```

```
axes[0,1].set_ylabel('Surge Multiplier')
axes[0,1].set_xticks(range(0,24,2))
axes[0,1].legend()
axes[0,1].grid(axis='y', alpha=0.3)

tier_colors = ['#1f77b4','#ff7f0e','#2ca02c']
bars3 = axes[1,0].bar(fare_by_tier.index, fare_by_tier['avg_fare_per_mi'],
                      color=tier_colors, edgecolor='black', alpha=0.85)
axes[1,0].set_title('Average Fare per Mile by Distance Tier',
fontweight='bold')
axes[1,0].set_xlabel('Distance Tier')
axes[1,0].set_ylabel('Fare per Mile ($)')
axes[1,0].tick_params(axis='x', rotation=10)
for bar, val in zip(bars3, fare_by_tier['avg_fare_per_mi']) :
  axes[1,0].text(bar.get_x()+ bar.get_width()/2, bar.get_height() + 0.1,
                 f'${val:.2f}', ha='center', fontsize=10,
fontweight='bold' )
axes[1,0].grid(axis='y', alpha=0.3)

bars4 = axes[1,1].bar(vendor_stats.index, vendor_stats['avg_fare'],
                color=['#9467bd','#8c564b'], edgecolor='black',
alpha=0.85, width=0.4)
axes[1,1].set_title('Average Fare by Vendor', fontweight='bold' )
axes[1,1].set_xlabel('Vendor')
axes[1,1].set_ylabel('Average Fare ($)')
for bar, val in zip(bars4, vendor_stats['avg_fare']):
  axes[1,1].text(bar.get_x()+ bar.get_width()/2, bar.get_height() +0.05,
                 f'${val:.2f}', ha='center', fontsize=11,
fontweight='bold')
axes[1,1].grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()

total_current = (fare_by_hour['avg_fare' ] *
fare_by_hour['trip_count']).sum()
total_proposed = (fare_by_hour['proposed_fare'] *
fare_by_hour['trip_count']) .sum()
uplift_pct = (total_proposed - total_current) / total_current * 100

print(df_zones.shape)

print(f"\nESTIMATED REVENUE IMPACT OF PROPOSED PRICING:")
```

```
print(f"  Current  revenue (sample)  : ${total_current:>12,.2f}")
print(f"  Proposed revenue (sample)  : ${total_proposed:>12,.2f}")
print(f"  Estimated uplift           : +{uplift_pct:.1f}%")
print(f"\n  → Applying surge pricing selectively during peak hours")
print(f"    can improve revenue by ~{uplift_pct:.0f}% without increasing
trip volume.")
```

Analysis:

1. Implement time-based surge pricing: A demand-responsive multiplier system is strongly justified by the data. Proposed multipliers — Evening Peak (5-8 PM): 1.30x base fare; Morning Rush (7-10 AM): 1.20x; Late Night (10 PM-midnight): 1.15x; Early Morning (2-6 AM, low demand): 0.85x off-peak discount to stimulate demand. These adjustments are estimated to improve per-hour revenue by 15-30% during peak windows while the off-peak discount encourages trips during underutilised hours.

2. Revise short-trip pricing to close the vendor gap: Vendor 1 (CMT) charges only $9.89/mile for 0-2 mile trips versus Vendor 2 (VTS) at $18.10/mile — an 83% gap. CMT can competitively increase its short-trip rate to $12-14/mile while remaining below VTS, capturing higher revenue on the 273,382 short CMT trips without losing customers to cash alternatives.

3. Promote longer, higher-value trips: The fare trend shows $3.10 earned per additional mile, while average fare for 5+ mile trips is $48-51 — substantially more than shorter rides. Offering incentives for airport and outer-borough trips (e.g., priority dispatch, reduced waiting time) can shift the trip mix toward these higher-value routes.

4. Leverage the tip stability: With a consistent 15.4% tip rate among credit card users, pricing changes should not be designed to increase tips (already near ceiling) but rather to increase base fares. Displaying suggested tip amounts (15%, 20%, 25%) on screens rather than a custom entry further normalises higher tip selections.

5. Address the cash payment gap: Cash trips ($25.21 avg fare) earn less than credit card trips ($29.88 avg fare), partly because tips are not captured. Incentivising card payment through small discounts or rewards could shift the 17.2% cash segment, capturing tip revenue currently off the books.